

Python on the IBMi- Chapter 7



Our hero now stands before the dragon, not to conquer it, but to make a choice. Deletion is a powerful force, and once unleashed, there's no turning back. Every decision must be deliberate, every strike precise, for one wrong move could bring unintended consequences.

In this chapter, we'll add the ability to delete records from the Product Master. When a user clicks the delete icon, a confirmation prompt will appear: *"Are you sure you want to delete this product?"* If confirmed, a backend RPG program will attempt the deletion, and a notification will report whether the operation succeeded or failed. It's a simple task on the surface, but like any true dragon, it demands respect.

Disclaimer

This article will use the built-in Python HTTP Server. While this works, it is NOT production ready. While it might work for simple utilities and very low demand. It's a very simple HTTP Server that handles basic requests. The next few articles are for demonstration purposes only.

Prerequisites

- Please refer to chapter 1 for other prerequisites (<https://www.linkedin.com/pulse/python-ibmi-chapter-1-ricky-thompson-zg7te>)
- Code can be found here: (https://github.com/thomprl/ibmi_python_rpg_adventures)

Objective

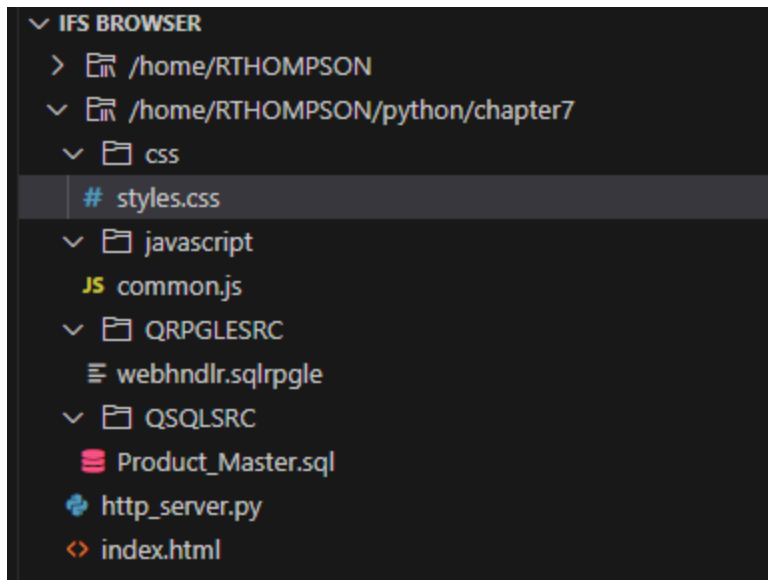
- Expand on Chapter 6 and add the ability to delete records from our Product Master using a POST request.
- Enhance our Python Web Server Code to handle a HTTP POST method
- Modify our RPG program to include deleting a record.

Before We Begin

Again, there will be a good amount of code used in this article. Most of it is repeated with updates from the previous chapters but it will be hard to explain in detail what each line of code is doing.

- RPG program (WEBHNDLR) we will add the delete operation and some other small tweaks
- Python script (http_server.py) we will add the ability to handle the HTTP Post method to be passed to the RPG program
- HTML (index.html) numerous changes to the index HTML file to add icons for delete and a future edit icon. Add code to send the delete request to the Python Server. Add a notification to display any success/error messages we might encounter. A new menu navigation bar to make things cleaner.
- Create a new CSS file called (**styles.css**) this will be used to help format our navigation bar, icons, and notification messages. This is also to help keep our index.html smaller and more manageable. This will be placed in its own folder (**/home/rthompson/chapter7/css**).
- Change our (**common.js**) JavaScript file to forward the delete request to the web server and handle showing a notification message on the delete

This might come in handy as well. Each article I make a new IFS folder and put all the contents into it. This is a picture of my (**/home/rthompson/python/chapter7**) folder from VS Code.



Let's Code

Let's start with the RPG changes. I'll add code to handle the HTTP Post method and a new sub-procedure to handle the delete request. When the method = POST it will call the DeleteProduct sub-procedure and attempt to delete the record. We will return a success or error message letting the user know if it was successful or not.

webhndlr.sqlrpgle

```
**free
Ctl-Opt option(*nodebugio:*srcstmt:*nounref) Main(Main)
dftactgrp(*no)
  text('Web Server RPG Handler Program');

// The main procedure that will listen for incoming requests
Dcl-Proc Main;
  Dcl-Pi *N;
    pMethod CHAR(10) const;
    pRequestData CHAR(32000) const;
    pResponseData CHAR(32000);
  End-Pi;

  Dcl-S vJsonString CHAR(32000);
  Dcl-S vApp VARCHAR(50);
  Dcl-S vAction VARCHAR(50);
  // Dcl-S vRequestData SQLTYPE(CLOB:500000);

  Exec Sql
    Set Option commit = *none;
```

```

// If Request Data is blank return null
If pRequestData = '';
    pResponseData = '{"data": null}';
    Return;
Else;
    Exec Sql
        SELECT upper(jt.app)
            ,upper(jt.action)
        INTO :vApp
            ,:vAction
        FROM JSON_TABLE(trim(:pRequestData),
            'lax $'
            COLUMNS (
                app VARCHAR(50) PATH 'lax $.app',
                action VARCHAR(50) PATH 'lax $.action'
            )
        ) AS jt;

    If sqlcod <> 0;
        pResponseData = '{ "success": false, "message": "Invalid
JSON format" }';
        Return;
    EndIf;
EndIf;

If %upper(pMethod) = 'GET ';
    Select;
        When vApp = 'PRODUCT' and vAction = 'GETPRODUCTLIST';
            // Call the procedure to get the Product List in JSON
format
            vJsonString = GetProductList();
            pResponseData = vJsonString;
        Other;
            pResponseData = '{"success": false, "message": "Unknown
Action Sent"}';
        EndSl;
    EndIf;

If %upper(pMethod) = 'POST';
    Select;
        When vApp = 'PRODUCT' and vAction = 'DELETE';
            vJsonString = DeleteProduct(pRequestData);
            pResponseData = vJsonString;
        Other;
            pResponseData = '{"success": false, "message": "Unknown
Action Sent"}';
        EndSl;

```

```

    EndIf;

End-Proc;

Dcl-Proc GetProductList;
    Dcl-Pi *N CHAR(32000);
    End-Pi;

    Dcl-S jsonString CHAR(32000);
    Dcl-S jsonStringClob SQLTYPE(CLOB:32000);

    Exec Sql
    Select Cast(
        Json_Object(
            'data' Value (Json_Arrayagg(
                Json_Object(
                    'ProductNumber' Value rtrim(product_number)
                    , 'Description' Value Rtrim(description)
                    , 'Cost' Value Cost
                    , 'UnitOfMeasure' Value Rtrim(unit_of_measure)
                    , 'Category' Value Rtrim(category)
                )
            ))
        ) As Clob(32000))
    Into :jsonstringclob
    From rthompson1.product_master;

    JsonString = JsonStringClob_Data;
    Return JsonString;
End-Proc;

Dcl-Proc DeleteProduct;
    Dcl-Pi *N CHAR(32000);
    pRequestData CHAR(32000) const;
    End-Pi;

    Dcl-S vProductNumber VARCHAR(50);
    Dcl-S vJsonResponse CHAR(32000);

    Exec Sql
    SELECT jt.productnumber
    INTO :vProductNumber
    FROM JSON_TABLE(trim(:pRequestData),
        'lax $'
        COLUMNS (
            productnumber VARCHAR(20) PATH 'lax $.productnumber'
        )
    )

```

```

    ) AS jt;

    If sqlcod <> 0;
        vJsonResponse = '{"success": false, "message": "Invalid JSON
format"}' ;
        Return vJsonResponse;
    EndIf;

    // Perform the delete operation
    Exec Sql
        DELETE FROM rthompson1.product_master
        WHERE product_number = :vProductNumber;

    If sqlcod = 0;
        vJsonResponse = '{"success": true, "message": "Successfully
deleted product: '
            + %trim(vProductNumber) + '" }';
    Else;
        vJsonResponse = '{"success": false, "message": "Failed to
delete product: '
            + %trim(vProductNumber) + '" }';
    EndIf;

    Return vJsonResponse;
End-Proc;

```

styles.css

Let's create a stylesheet to help format our navigation bar, icons, and notification message. This is placed in **(/home/rthompson/python/chapter7/css)**.

```

.action-buttons {
    white-space: nowrap;
}

.action-buttons i {
    margin: 0 5px;
    cursor: pointer;
}

.fa-trash-alt:hover {
    color: red;
}

.fa-edit:hover {
    color: blue;
}

```

```
}

#notification {
    position: fixed;
    top: 60px;
    left: 50%;
    transform: translateX(-50%);
    min-width: 300px;
    max-width: 80%;
    padding: 15px;
    border-radius: 4px;
    box-shadow: 0 4px 6px rgba(0, 0, 0, 0.1);
    display: none;
    z-index: 900;
}

#notification.over-modal {
    z-index: 1100;
    /* Above modal when needed */
    top: 25%;
    /* Center vertically */
    transform: translate(-50%, -50%);
    /* Center both horizontally and vertically */
}

#notification.success {
    background-color: #e8f5e9;
    border: 1px solid #66bb6a;
    color: #2e7d32;
}

#notification.error {
    background-color: #ffebee;
    border: 1px solid #ef5350;
    color: #c62828;
}

/* Navigation bar styles */
.navbar {
    overflow: hidden;
    background-color: #333;
}

.navbar a {
    float: left;
    font-size: 16px;
    color: white;
```

```
    text-align: center;
    padding: 14px 16px;
    text-decoration: none;
}

.dropdown {
    float: left;
    overflow: hidden;
}

.dropdown .dropbtn {
    font-size: 16px;
    border: none;
    outline: none;
    color: white;
    padding: 14px 16px;
    background-color: inherit;
    font-family: inherit;
    margin: 0;
}

.navbar a:hover,
.dropdown:hover .dropbtn {
    background-color: rgb(5, 33, 245);
}

.dropdown-content {
    display: none;
    position: absolute;
    background-color: #f9f9f9;
    min-width: 160px;
    box-shadow: 0px 8px 16px 0px rgba(0, 0, 0, 0.2);
    z-index: 1;
}

.dropdown-content a {
    float: none;
    color: black;
    padding: 12px 16px;
    text-decoration: none;
    display: block;
    text-align: left;
}

.dropdown-content a:hover {
    background-color: #ddd;
}
```



```
.dropdown:hover .dropdown-content {
    display: block;
}
```

common.js

We need to add to our **(common.js)** file to forward our delete request to the web server and also the ability to show the notification window to show success or failures. The notification message will disappear after 10 secs. This file is located in **(/home/rthompson/python/chapter7/javascript).**

```
$(document).ready(function () {
    var table = $('#myTable').DataTable({
        ajax: {
            url: '/api?app=product&action=getproductlist',
            type: 'GET',
            dataSrc: 'data'
        },
        columns: [
            { 'data': 'ProductNumber' },
            { 'data': 'Description' },
            { 'data': 'Cost' },
            { 'data': 'UnitOfMeasure' },
            { 'data': 'Category' },
            {
                'data': null,
                'render': function (data, type, row) {
                    return `
                        <div class="action-buttons">
                            <i class="fas fa-edit"
title="Edit"></i>
                                <i class="fas fa-trash-alt"
onclick="deleteProduct('${row.ProductNumber}')"
title="Delete"></i>
                            </div>
                        `;
                }
            },
        ],
        pageLength: 10,
        order: [[1, 'asc']]
    });
});

function showNotification(message, type) {
```

```

        const notification =
document.getElementById('notification');
        const messageElement =
document.getElementById('notification-message');
        notification.className = type;
        messageElement.textContent = message;
        notification.style.display = 'block';

        // Auto-hide after 10 seconds
        setTimeout(hideNotification, 10000);
    }

function hideNotification() {
    document.getElementById('notification').style.display =
'none';
}

function deleteProduct(productNumber) {
    if (confirm(`Are you sure you want to delete product:
${productNumber}?`)) {
        $.ajax({
            url: '/api',
            type: 'POST',
            contentType: 'application/json',
            data: JSON.stringify({
                app: 'product',
                action: 'delete',
                productnumber: productNumber
            }),
            success: function (response) {
                if (response.success) {
                    $('#myTable').DataTable().ajax.reload();
                    showNotification(response.message,
'success');
                } else {
                    showNotification(response.message, 'error');
                }
            },
            error: function (xhr, status, error) {
                showNotification('Error deleting product: ' +
error, 'error');
            }
        });
    }
}

```

http_server.py

We add the ability to handle a POST request to our Python script (**http_server.py**) file. It will forward the request just like the GET request. It will format the request data into JSON to pass on to the RPG.

```
# Import the built-in HTTP server class for handling web
requests
from http.server import HTTPServer, SimpleHTTPRequestHandler
from urllib.parse import urlparse, parse_qs

# Import argparse for command-line argument parsing
import argparse
# Import standard libraries for handling JSON
import json
import threading

# Import IBM DB2 connection tools and itoolkit
import ibm_db
from itoolkit import *
from itoolkit.transport import DirectTransport

# Create a transport object that allows the toolkit to talk to
IBM i using SQL
itransport = DirectTransport()
print("Starting HTTP Server... Please wait...")

# -----
# DEFINE HTTP REQUEST HANDLER CLASS
# -----

class RPGHTTPRequestHandler(SimpleHTTPRequestHandler):
    LIBRARY = 'RTHOMPSON1' # default; will be overridden

    """
    Custom HTTP handler that serves static files and also
    handles /api GET and POST requests,
    calling an RPGLE program using itoolkit.
    """

    def call_rpg(self, method, req_data):
        """
        Call the RPGLE program 'WEBSERV' in library,
        passing in the HTTP method and request data as
        parameters,
        and returning the RPG response.
```

```

Parameters:
    method (str): 'GET' or 'POST'
    req_data (str): JSON or text from the HTTP request

Returns:
    str: response string returned from RPG program
"""

# Initialize the toolkit object
itool = iToolkit()

# Add the library to the library list so the program can
be found
itool.add(iCmd('addlible', f'ADDLIB {self.LIBRARY}'))

# Add the RPG program call with 3 parameters:
# - method: 10-character input
# - request: 32000-character input
# - response: 32000-character output (blank string
placeholder)
itool.add(iPgm('pgmcall', 'WEBHNDLR')
        .addParm(iData('method', '10a', method))
        .addParm(iData('request', '32000a', req_data))
        .addParm(iData('response', '32000a', ' '))
        )

# Execute the RPG call
itool.call(itransport)

# Retrieve the output from the RPG program
result = itool.dict_out('pgmcall')

# Return the response value or fallback if missing
return result.get('response', 'No response')

# Handle HTTP GET requests to /api by calling the RPG
program
def do_GET(self):
    if self.path.startswith('/api'):

        # Parse query string into JSON string
        query = urlparse(self.path).query
        query_dict = parse_qs(query)

        # Check for shutdown command

```

```

        if query_dict.get('action', [''])[0].upper() ==
'SHUTDOWN':
            print("⚠ Shutdown requested via HTTP")

            def stop_server():
                print("▼ Shutting down server...")
                self.server.shutdown()

            threading.Thread(target=stop_server).start()

            self.send_response(200)
            self.send_header('Content-Type',
'application/json')
            self.end_headers()
            self.wfile.write(json.dumps({"status": "Server
shutting down"}).encode())
            return

        #Normal GET request processing
        query_json = json.dumps({k: v[0] for k, v in
query_dict.items()})

        request_data = query_json
        print("=== Incoming GET request ===")
        print(f"Query string: {query_json}")

        try:

            # Call the RPG program with method = 'GET'
            rpg_result = self.call_rpg('GET', request_data)
            print(f"Return Data: {rpg_result}")

            # Return the result to the client
            self.send_response(200)
            self.send_header('Content-Type',
'application/json')
            self.end_headers()
            self.wfile.write(rpg_result.encode())

        except Exception as e:
            print("=== Incoming GET request ===")
            print(f"Query string: {query_json}")

            print("=== RPG Response (raw) ===")
            print(rpg_result)

            print("ERROR during RPG call:", e)

```

```

        self.send_response(500)
        self.end_headers()
        self.wfile.write(json.dumps({"error":
str(e)}).encode())
    else:
        # Serve regular static files (like index.html, etc.)
        super().do_GET()

    # Handle HTTP POST requests to /api by calling the RPG
program
    def do_POST(self):
        if self.path.startswith('/api'):
            content_length = int(self.headers.get('Content-
Length', 0))
            content_type = self.headers.get('Content-Type', '')

            # Read the POST data
            post_data =
self.rfile.read(content_length).decode('utf-8')

            print("=== Incoming POST request ===")
            print(f"Content-Type: {content_type}")
            print(f"Raw POST data: {post_data}")

            try:
                # If content is JSON, parse it
                if 'application/json' in content_type:
                    # Parse JSON data
                    json_data = json.loads(post_data)
                    # Convert back to string for RPG program
                    request_data = json.dumps(json_data)
                else:
                    request_data = post_data

                print(f"Processed request data: {request_data}")

                # Call the RPG program with method = 'POST'
                rpg_result = self.call_rpg('POST', request_data)
                print(f"Return Data: {rpg_result}")

                # Return the result to the client
                self.send_response(200)
                self.send_header('Content-Type',
'application/json')
                self.end_headers()
                self.wfile.write(rpg_result.encode())

```

```

        except json.JSONDecodeError as e:
            print("ERROR parsing JSON:", e)
            self.send_response(400)
            self.send_header('Content-Type',
'application/json')
            self.end_headers()
            self.wfile.write(json.dumps({
                "success": False,
                "message": f"Invalid JSON format: {str(e)}"
            }).encode())

        except Exception as e:
            print("ERROR during RPG call:", e)
            self.send_response(500)
            self.send_header('Content-Type',
'application/json')
            self.end_headers()
            self.wfile.write(json.dumps({
                "success": False,
                "message": f"Error processing request:
{str(e)}"
            }).encode())

# -----
# START THE SERVER
# -----

if __name__ == '__main__':
    # Setup argument parser
    parser = argparse.ArgumentParser(description='Simple RPG
HTTP Server')
    parser.add_argument('--port', type=int, default=43001,
help='Port to listen on')
    parser.add_argument('--library', type=str,
default='RTHOMPSON1', help='Library to add to library list')

    args = parser.parse_args()

    # Save the library name globally
    RPGHTTPRequestHandler.LIBRARY = args.library

    # Start the HTTP server
    port = args.port
    server = HTTPServer(('0.0.0.0', port),
RPGHTTPRequestHandler)
    print(f"Serving on http://localhost:{port} using library
{args.library}")
    server.serve_forever()

```

index.html

We add the notification window, the stylesheet, and the edit/delete icons to the index.html.

```
<!DOCTYPE html>
<html>

<head>
  <meta charset="UTF-8">
  <title>Product List</title>

  <script
src="https://ajax.googleapis.com/ajax/libs/jquery/3.7.1/jquery.m
in.js"></script>
  <link href="https://cdn.datatables.net/v/dt/dt-
2.3.2/datatables.min.css" rel="stylesheet"
    integrity="sha384-
d76uxpdVr9QyCSR9vVSYdOAZeRzNUN8A4JVqUHBVXyGxZ+oOfrZVHC/1Y58mhyNg
" crossorigin="anonymous">
  <script src="https://cdn.datatables.net/v/dt/dt-
2.3.2/datatables.min.js"
    integrity="sha384-
JRUjeYWWUGO171YFugrU0ksSC6CaWnl4XzwP6mNjnnDh4hfFGRyYbEXwryGwLsEp
"
    crossorigin="anonymous"></script>
  <link rel="stylesheet"
href="https://cdnjs.cloudflare.com/ajax/libs/font-
awesome/6.5.1/css/all.min.css">
  <link rel="stylesheet" href="css/styles.css">
  <script src="javascript/common.js"></script>

</head>

<body>
  <!-- Navigation Bar -->
  <div class="navbar">
    <a href="/">Home</a>
    <div class="dropdown">
      <button class="dropbtn">Tools
        <i class="fa fa-caret-down"></i>
      </button>
      <div class="dropdown-content">
        <a
href="/api?app=product&action=getproductlist">Get Raw JSON
Product List</a>
```



```

        <a href="/api?action=shutdown">Shutdown Web
Server</a>
    </div>
</div>
<br><br>
<div id="notification" style="display:none; padding: 10px;
margin: 10px; border-radius: 4px;">
    <span id="notification-message"></span>
    <button onclick="hideNotification()"
style="float:right;">&times;</button>
</div>

<br><br>
<table id="myTable" class="display table table-striped
table-bordered" style="width:100%">
    <thead>
        <tr>
            <th>Product Number</th>
            <th>Description</th>
            <th>Cost</th>
            <th>UOM</th>
            <th>Category</th>
            <th>Actions</th>
        </tr>
    </thead>
</table>
</body>

</html>

```

Start the Web Server

Make sure the port you're using (e.g., 43001) is available on your IBMi.

From VS Code Open a PASE terminal by pressing (**Ctrl+Shift+J**) and selecting PASE and then type the following:

```
cd python/chapter7
```

```
python3 http_server.py --port 43001 --library RTHOMPSON1
```

It will take a few seconds to start the server. You will see a message in the terminal similar to this.

```
-bash-5.2$ python3 http_server.py --port 43001 --library RTHOMPSON1
Starting HTTP Server... Please wait...
Serving on http://localhost:43001 using library RTHOMPSON1
```

Now open a web browser and go to your IBMi name or IP address plus the port number.

`http://myibmi:43001/`

You should see web page open with a list of products being returned from our RPG program. Now attempt to delete a product. You will get a Are you sure message. If you click Ok it will attempt to delete the product and return a notification message if it was successful or not. After 10 seconds the message will disappear or you can close it with by pressing X.

Home Tools

Successfully deleted product: P0005

10 entries per page

Search:

Product Number	Description	Cost	UOM	Category	Actions
P0013	AA Batteries (4 Pack)	4.25	PACK	Electronics	
P0008	Ballpoint Pens (Pack of 10)	2.99	PACK	Stationery	
P0002	Bluetooth Keyboard	29.95	EACH	Electronics	
P0006	Coffee Mug	5.25	EACH	Kitchen	
P0009	Desk Lamp	18.75	EACH	Office	
P0020	Electric Kettle	24.5	EACH	Kitchen	
P0004	Laptop Backpack	45	EACH	Accessories	
P0016	LED Light Bulbs (6 Pack)	12.49	PACK	Home Improvement	
P0019	Mechanical Pencil Set (2 Pack)	4.75	PACK	Stationery	
P0007	Notebook - Lined	3.15	EACH	Stationery	

Showing 1 to 10 of 19 entries

1 2

What's Next

With the dragon behind him, our hero now faces a new enemy, a dark knight who doesn't wield fire, but tests of accuracy and discipline. This battle isn't just about brute strength; it's about making sure every strike lands true.

In Chapter 8, we'll take on the challenge of editing Product Master records. When the edit button is clicked, a modal will appear, allowing changes to be made. Before any update is accepted, we'll validate that each field contains the correct information. Victory in this chapter will come not from power, but from precision.